

Lecture 2 - Kinematics and Behavioral robotics

Mobile Robotics (ELA408)

Lennie Carlén Eriksson

Mälardalen University

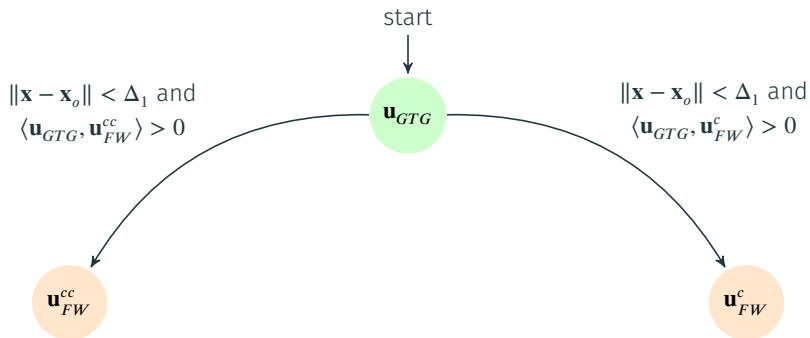
lennie.carlen.eriksson@mdu.se

Version: April 6, 2026

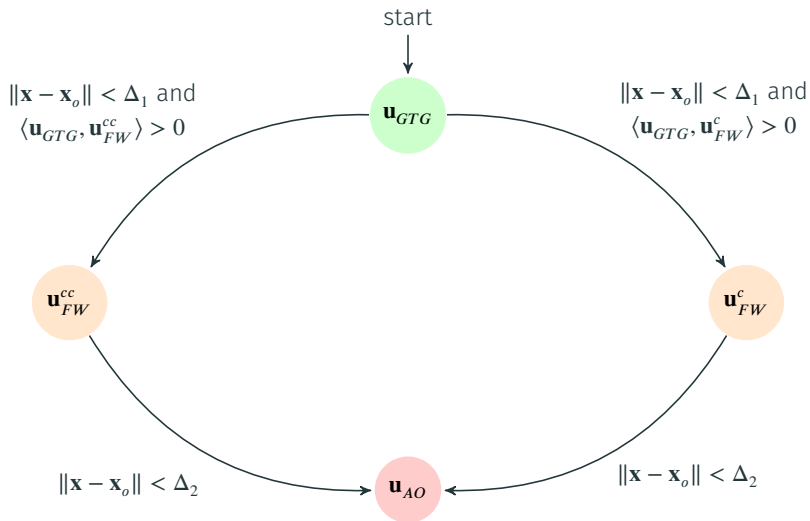
Introduction – Outline

1. Kinematic Models
2. Other Robot Classes
3. Car-Like Robots

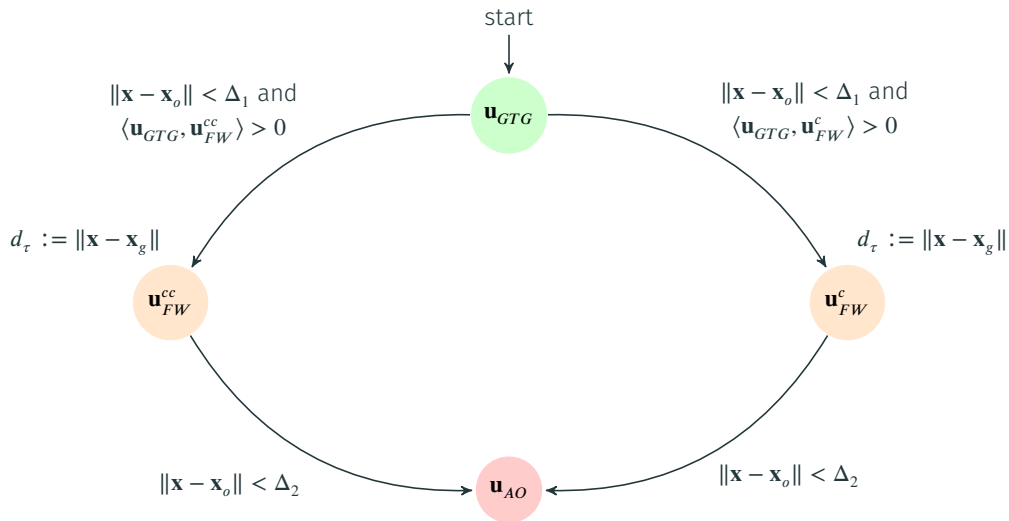
Putting it all together



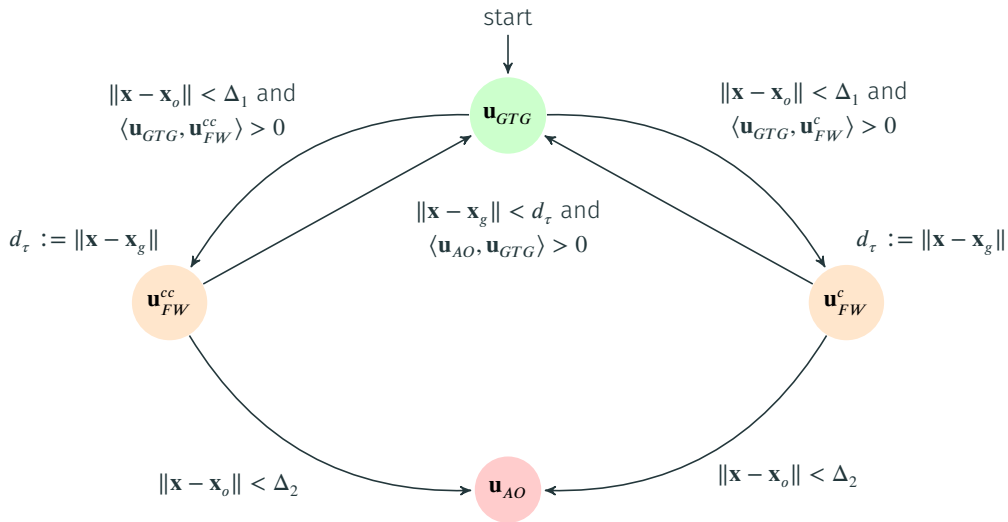
Putting it all together



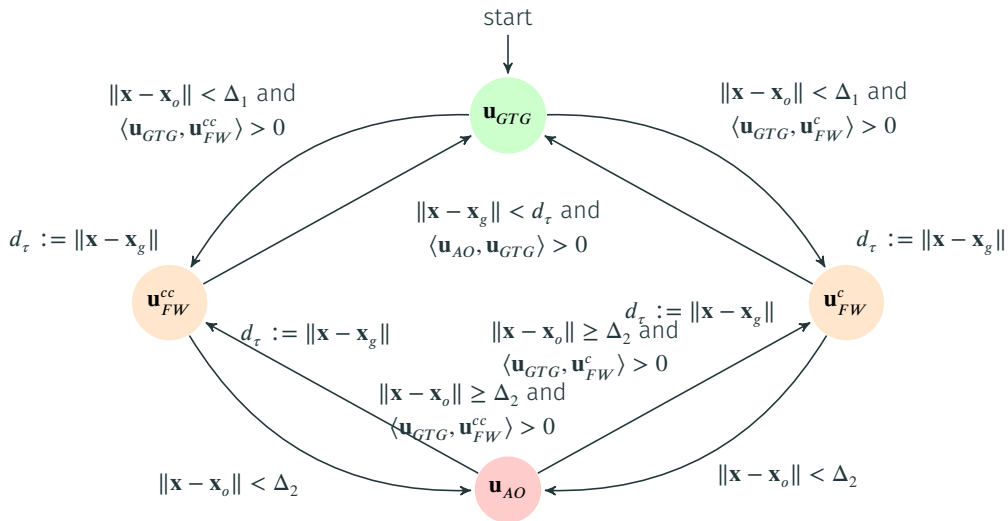
Putting it all together



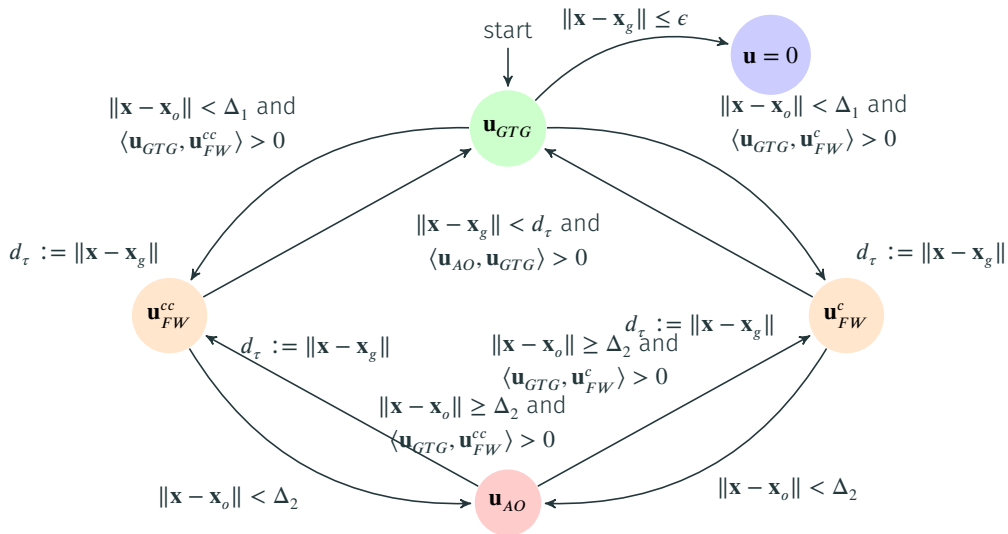
Putting it all together



Putting it all together



Putting it all together



Kinematic Models

Kinematic Models

Planning

A layered architecture

How do we make a unicycle robot “act” like $\dot{\mathbf{x}} = \mathbf{u}$?

- We have a problem: Even with a simple robot model ($\dot{\mathbf{x}} = \mathbf{u}$), the navigation architecture becomes rather involved
- Would like to be able to **reuse** this while allowing for **more realistic robot models**
 - Car-like robots
 - Snake robots
 - Quadrotors
 - ...

All Good Things Come in Threes

- Standard navigation systems are typically decoupled along three different levels of abstraction:
 1. **Strategic Level:** Where to go (high-level, long-term)?
 - The goals are defined at this level

All Good Things Come in Threes

- Standard navigation systems are typically decoupled along three different levels of abstraction:
 1. **Strategic Level**: Where to go (high-level, long-term)?
 - The goals are defined at this level
 2. **Operational Level**: Where to go (low-level, short-term)?
 - Where do I go now to reach the goal?
 - $\dot{\mathbf{x}} = \mathbf{u}$ lives at this level

All Good Things Come in Threes

- Standard navigation systems are typically decoupled along three different levels of abstraction:
 1. **Strategic Level**: Where to go (high-level, long-term)?
 - The goals are defined at this level
 2. **Operational Level**: Where to go (low-level, short-term)?
 - Where do I go now to reach the goal?
 - $\dot{\mathbf{x}} = \mathbf{u}$ lives at this level
 3. **Tactical Level**: How to go there?
 - Mapping the $\dot{\mathbf{x}} = \mathbf{u}$ into the real robot is at this level

All Good Things Come in Threes

- Slightly less militaristic:
 1. **High-Level Planning**: Where should the (intermediary) goal points be?
 2. **Low-Level Planning**: Which “direction” to move in-between goal points?
 3. **Execution**: How make the robot move in those directions?

All Good Things Come in Threes

- Slightly less militaristic:
 1. **High-Level Planning**: Where should the (intermediary) goal points be?
 - We will speak about this at the end of the course
 2. **Low-Level Planning**: Which “direction” to move in-between goal points?
 3. **Execution**: How make the robot move in those directions?

All Good Things Come in Threes

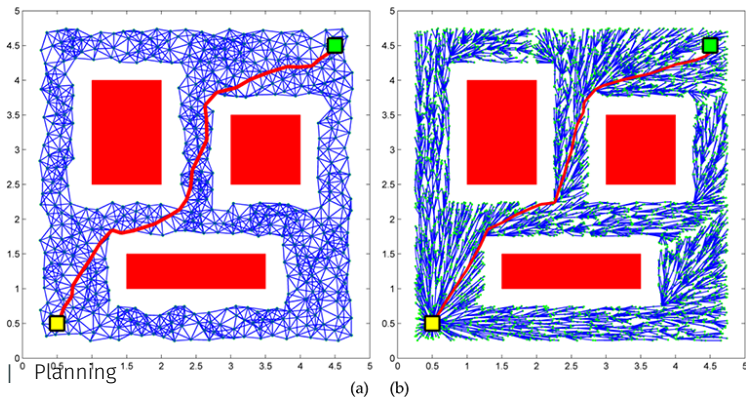
- Slightly less militaristic:
 1. **High-Level Planning**: Where should the (intermediary) goal points be?
 - We will speak about this at the end of the course
 2. **Low-Level Planning**: Which “direction” to move in-between goal points?
 - Use the navigation architecture that we designed
 3. **Execution**: How make the robot move in those directions?

All Good Things Come in Threes

- Slightly less militaristic:
 1. **High-Level Planning**: Where should the (intermediary) goal points be?
 - We will speak about this at the end of the course
 2. **Low-Level Planning**: Which “direction” to move in-between goal points?
 - Use the navigation architecture that we designed
 3. **Execution**: How make the robot move in those directions?
 - Control design with reference signal (generated by the low-level planning)

High-level Planning

- There are many AI methods (e.g., Dijkstra, Dynamic Programming, A*, D*, RRT, ...) for doing this
- We will see few of them
 - Discretize the world with cells
 - Graph-based methods
 - ...



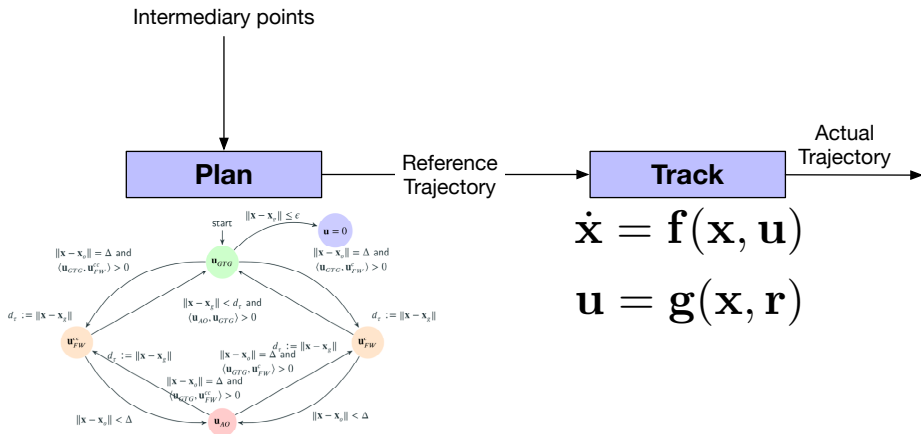
Low-level planning

- We already know how to do this!
- Assume that $\dot{\mathbf{x}} = \mathbf{u}$, $\mathbf{x}, \mathbf{u} \in \mathbb{R}^2$ and get to work!
- The “output” is a **desired direction** (and magnitude) of travel

Execution-level

- This is where we make the unicycle (or any other mobile robot) act like a simpler system over which we are performing the low-level planning

The Layered architecture



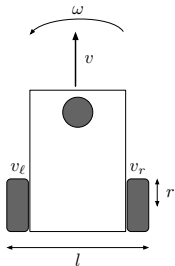
Kinematic Models

Differential Drive Trackers

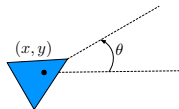
Differential-Drive Trackers

- How should we design the tracker when the robot is a differential-drive mobile robot?

Recap: The Model



$$\begin{cases} \dot{x} = \frac{r}{2}(v_r + v_\ell) \cos \theta \\ \dot{y} = \frac{r}{2}(v_r + v_\ell) \sin \theta \\ \dot{\theta} = \frac{r}{l}(v_r - v_\ell) \end{cases}$$



$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

The connection between them is:

$$\begin{cases} v_r = \frac{2v + \omega l}{2r} \\ v_\ell = \frac{2v - \omega l}{2r} \end{cases}$$

Recap: Dealing with angles

- How drive the robot in a specific direction θ_d ?
- The considered model is:

$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

- We solved that by:

$$e = \text{atan2}(\sin(\theta_d - \theta), \cos(\theta_d - \theta)), \quad \omega = \text{PID}(e)$$

$$\text{PID}(e) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \dot{e}(t)$$

Designing a tracker: Adding in the speed component

- Let the output from the planner be $\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$ ($\dot{\mathbf{x}} = \mathbf{u}$)
- Why don't we set the velocity to be equal to $\mathbf{u}$? We can set

$$\theta_d = \text{atan2} \left(\frac{u_2}{u_1} \right)$$

- As for the velocity component we can notice that:

$$\sqrt{\dot{x}^2 + \dot{y}^2} = \sqrt{v^2 \cos^2 \theta + v^2 \sin^2 \theta} = v$$

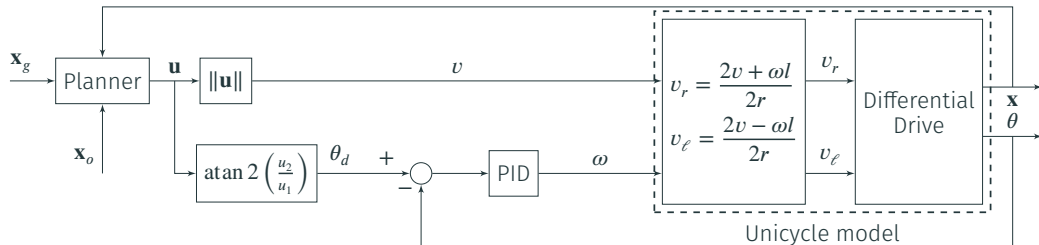
therefore we can just set:

$$v = \|\mathbf{u}\| = \sqrt{u_1^2 + u_2^2}$$

The complete differential-drive architecture

1. We get (u_1, u_2) from the low-level plan
2. We compute the $\theta_d = \text{atan2}(u_2/u_1)$
3. We compute $\omega = \text{PID}(\theta_d - \theta)$
4. We compute $v = \sqrt{u_1^2 + u_2^2}$
5. We obtain the actuation signal as

$$\begin{cases} v_r = \frac{2v + \omega l}{2r} \\ v_\ell = \frac{2v - \omega l}{2r} \end{cases}$$



Other Robot Classes

Other Robot classes

- It is indeed possible to make differential drive mobile robots “act” like $\dot{\mathbf{x}} = \mathbf{u}$
 - If we are willing to ignore orientation
 - And accept a small offset error
- Does this generalize to other types of robots? And, what other types are there?

Other models

- There are lots and lots of different types of robotic systems
- We cannot cover them all. Instead, we will focus on what they have in common
- For the unicycle model we have
 - States: Position and orientation
 - Inputs: Angular and translational velocities

Car-Like Robot

A Car-like robot

$$\dot{x} = v \cos(\theta + \gamma)$$

$$\dot{y} = v \sin(\theta + \gamma)$$

$$\dot{\theta} = \frac{v}{L} \sin(\gamma)$$

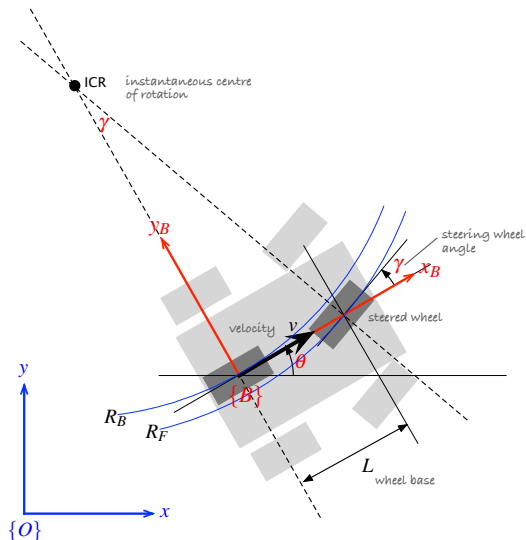
$$\dot{\gamma} = \sigma$$

- States:

- Position (x, y)
- Orientation θ
- Steering angle γ

- Inputs:

- Translational velocity: v
- Steering angular velocity: σ



Fixed-Wing Aircraft

A Fixed-Wing Aircraft

$$\dot{x} = v \cos(\theta)$$

$$\dot{y} = v \sin(\theta)$$

$$\dot{\theta} = \omega$$

$$\dot{z} = u$$

- States:
 - Position (x, y)
 - Orientation θ
 - Altitude z
- Inputs:
 - Translational velocity: v
 - Angular velocity: ω
 - Vertical velocity: u
- It kinds of behave like a unicycle
- It cannot hover (i.e., $v \neq 0$)



Underwater glider

An Underwater glider

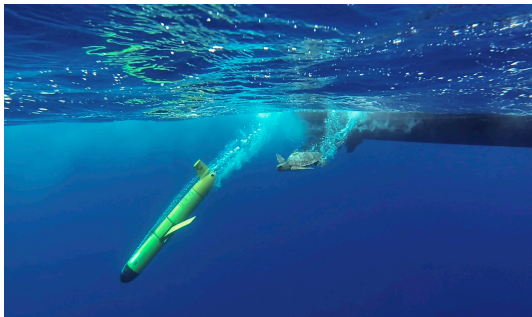
$$\dot{x} = v \cos(\theta)$$

$$\dot{y} = v \sin(\theta)$$

$$\dot{\theta} = \omega$$

$$\dot{z} = u$$

- States:
 - Position (x, y)
 - Orientation θ
 - Altitude z
- Inputs:
 - Translational velocity: v
 - Angular velocity: ω
 - Vertical velocity: u
- It has the same model as the fixed-wing aircraft



Summary

- Everything (almost) involves **POSE** = position (x, y) and heading θ !

Summary

- Everything (almost) involves **POSE** = position (x, y) and heading θ !
- Everything (almost) with a pose can be approximated as a unicycle model

Summary

- Everything (almost) involves **POSE** = position (x, y) and heading θ !
- Everything (almost) with a pose can be approximated as a unicycle model
- So we can (almost) use what we have already done and then make the actual model class fit the unicycle – Just add a layer
 - We will see this for the car robot

Adding Constraints

Many times we need constraints, which unfortunately make it harder to control the robots

Unicycle

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \omega$$

Dubins

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \omega$$

$$v = 1, \omega \in [-1, 1]$$

Reeds-Shepp

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

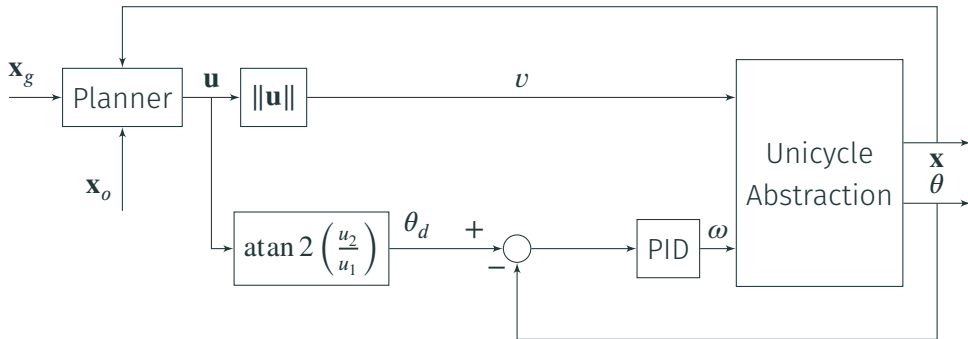
$$\dot{\theta} = \omega$$

$$|v| = 1, \omega \in [-1, 1]$$

- This constrains the type of motion
- The planning is more difficult
- Very famous model

- Dubins vehicle that can go backward
- Cannot be used for aircrafts

A general enough approach



- The only thing we need to do is to map (v, ω) into the specific control variables

When is POSE not reasonable?

- Humanoid
- Snakes
- Mobile manipulators

In all these applications, the pose is reasonable only for a sub-part of the robot, but not for the whole robot

Car-Like Robots

Car-Like Robots

- Claims
 - Pose (position and heading) is central
 - Other “pose-based” models can be made to look like a unicycle
- Now we focus on car-like robots

Car Kinematics

- What's different about the car is that it has four wheels
- Only the front wheels turn, which means that the steering wheel angle (front wheel angle) becomes important

States:

- (x, y) position
- θ heading
- γ steering angle

$$\dot{x} = v \cos(\theta + \gamma)$$

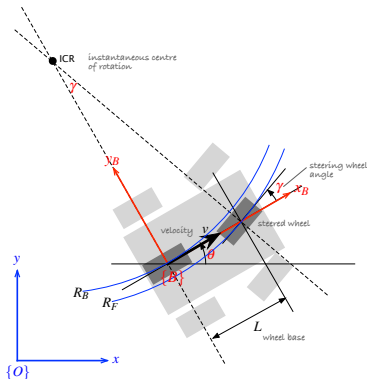
$$\dot{y} = v \sin(\theta + \gamma)$$

Inputs:

- v speed
- σ angular steering velocity

$$\dot{\theta} = \frac{v}{L} \sin(\gamma)$$

$$\dot{\gamma} = \sigma$$



Curvature Control: The Unicycle Case

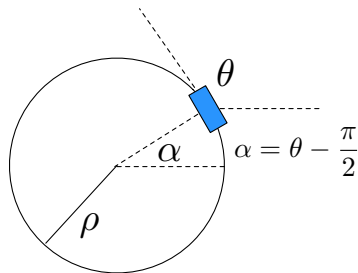
- How do we make this act like a unicycle?
- Assume a unicycle is driving along a circular arc

$$\dot{x} = v \cos(\theta + \gamma)$$

$$\dot{y} = v \sin(\theta + \gamma)$$

$$\dot{\theta} = \frac{v}{L} \sin(\gamma)$$

$$\dot{\gamma} = \sigma$$



$$\begin{aligned} x &= x_0 + \rho \cos \alpha \\ &= x_0 + \rho \sin \theta \end{aligned}$$

Curvature Control: The Unicycle Case

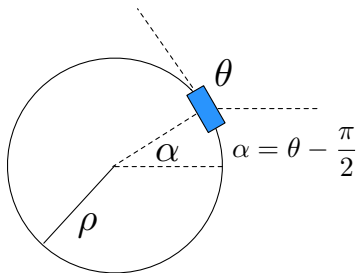
- How do we make this act like a unicycle?
- Assume a unicycle is driving along a circular arc

$$\dot{x} = v \cos(\theta + \gamma)$$

$$\dot{y} = v \sin(\theta + \gamma)$$

$$\dot{\theta} = \frac{v}{L} \sin(\gamma)$$

$$\dot{\gamma} = \sigma$$



$$x = x_0 + \rho \cos \alpha$$

$$= x_0 + \rho \sin \theta$$

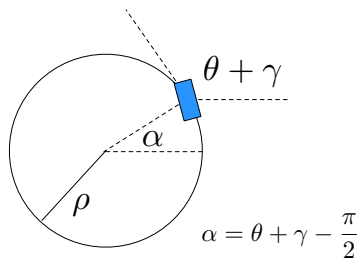
$$\dot{x} = \omega \rho \cos(\theta) = v \cos \theta$$

meaning that $\rho = \frac{v}{\omega}$
The curvature $\kappa = \frac{1}{\rho}$ and it is therefore

$$\kappa = \frac{\omega}{v}$$

Curvature Control: The Car case

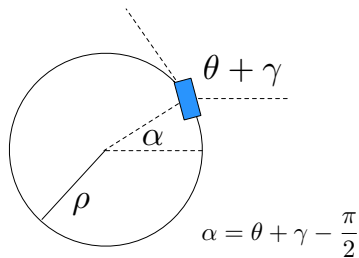
Let's redo this for the car



$$\begin{aligned}x &= x_0 + \rho \cos \alpha \\ &= x_0 + \rho \sin(\theta + \gamma)\end{aligned}$$

Curvature Control: The Car case

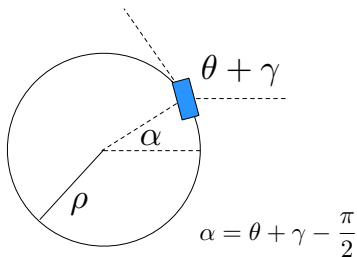
Let's redo this for the car



$$\begin{aligned}x &= x_0 + \rho \cos \alpha \\&= x_0 + \rho \sin(\theta + \gamma) \\ \dot{x} &= \rho(\dot{\theta} + \dot{\gamma}) \cos(\theta + \gamma)\end{aligned}$$

Curvature Control: The Car case

Let's redo this for the car

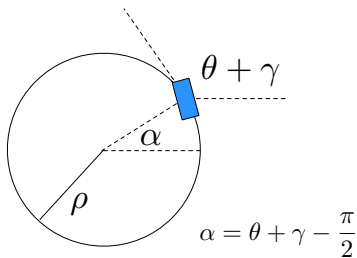


$$\begin{aligned}x &= x_0 + \rho \cos \alpha \\&= x_0 + \rho \sin(\theta + \gamma) \\ \dot{x} &= \rho(\dot{\theta} + \dot{\gamma}) \cos(\theta + \gamma)\end{aligned}$$

- If we are driving on a circle (no steering) then $\dot{\gamma} = 0$
- We have the equation for $\dot{\theta} = \frac{v}{L} \sin(\gamma)$

Curvature Control: The Car case

Let's redo this for the car

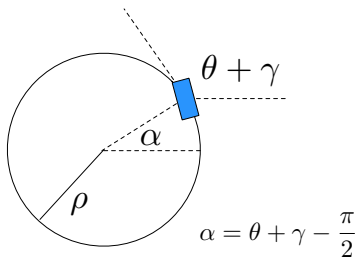


$$\begin{aligned}x &= x_0 + \rho \cos \alpha \\&= x_0 + \rho \sin(\theta + \gamma) \\ \dot{x} &= \rho(\dot{\theta} + \dot{\gamma}) \cos(\theta + \gamma)\end{aligned}$$

- If we are driving on a circle (no steering) then $\dot{\gamma} = 0$
- We have the equation for $\dot{\theta} = \frac{v}{L} \sin(\gamma)$
- We know also that $\dot{x} = v \cos(\theta + \gamma)$

Curvature Control: The Car case

Let's redo this for the car



$$\begin{aligned}x &= x_0 + \rho \cos \alpha \\&= x_0 + \rho \sin(\theta + \gamma) \\ \dot{x} &= \rho(\dot{\theta} + \dot{\gamma}) \cos(\theta + \gamma)\end{aligned}$$

- If we are driving on a circle (no steering) then $\dot{\gamma} = 0$
- We have the equation for $\dot{\theta} = \frac{v}{L} \sin(\gamma)$
- We know also that $\dot{x} = v \cos(\theta + \gamma)$
- The radius of the circle is $\rho = \frac{L}{\sin(\gamma)}$
- The curvature is therefore:

$$\kappa = \frac{\sin \gamma}{L}$$

Lining up the curvatures

- We got
 - Unicycle $\kappa = \frac{\omega}{v}$
 - Car: $\kappa = \frac{\sin \gamma}{L}$
- Combining them we get

$$\sin \gamma = \frac{\omega L}{v}$$

Lining up the curvatures

- We got
 - Unicycle $\kappa = \frac{\omega}{v}$
 - Car: $\kappa = \frac{\sin \gamma}{L}$
- Combining them we get

$$\sin \gamma = \frac{\omega L}{v}$$

- We cannot control the value of γ directly, but we can say that γ_d is the desired steering angle:

$$\gamma_d = \arcsin \left(\frac{\omega L}{v} \right)$$

But we can actually stay with the sinus instead of dealing with **arcsin**!

An “Almost P-Regulator”

- We could write something like this

$$\dot{\gamma} = \sigma$$

$$\sigma = K(\gamma_d - \gamma)$$

An “Almost P-Regulator”

- We could write something like this

$$\dot{\gamma} = \sigma$$

$$\sigma = K(\gamma_d - \gamma)$$

- But we can also write the same as:

$$\dot{\gamma} = \sigma$$

$$\sigma = K \left(\frac{\omega L}{v} - \sin(\gamma) \right)$$

Summary

